

Conceitos Fundamentais para RAG

Resumo conciso dos conceitos essenciais para sistemas de Retrieval-Augmented Generation.

1. Embeddings e Busca Semantica

Embeddings são representações vetoriais densas de texto em um espaço contínuo de alta dimensão. Cada palavra, frase ou documento é mapeado para um vetor numérico de modo que textos semanticamente próximos fiquem próximos nesse espaço (ex: 'gato' e 'felino' têm vetores similares).

O papel central dos embeddings em busca semântica é permitir a comparação por similaridade de cosseno em vez de correspondência exata de termos (keyword search). Enquanto uma busca booleana tradicional precisa que a palavra exata apareça no documento, a busca semântica com embeddings consegue recuperar documentos que tratam do mesmo assunto mesmo quando usam vocabulário diferente. Por exemplo, a query 'Como o computador aprende sozinho?' pode encontrar documentos sobre 'unsupervised learning' porque ambos geram vetores semanticamente próximos.

Modelos populares de embeddings incluem all-MiniLM-L6-v2 (leve, 384 dimensões), text-embedding-3-small (OpenAI, 1536 dimensões) e gemini-embedding-001 (Google). Quanto maior a dimensão, maior a capacidade de capturar nuances semânticas, mas também maior o custo computacional.

No contexto de RAG, os embeddings são usados em duas etapas: (1) no indexing, cada chunk do corpus é convertido em um vetor e armazenado em um banco vetorial (Chroma, FAISS, Pinecone); (2) no retrieval, a query do usuário é convertida no mesmo espaço vetorial e o banco retorna os K vizinhos mais próximos por similaridade de cosseno.

2. Chunking e sua Importancia

Chunking é o processo de dividir documentos longos em segmentos menores (chunks) antes de indexá-los. É uma etapa crucial porque: (a) modelos de embedding têm limite de tokens de entrada (tipicamente 256-512 tokens); (b) o contexto do LLM é limitado e chunks muito longos desperdiçam espaço; (c) chunks muito curtos fragmentam conceitos.

A escolha do tamanho do chunk é um trade-off: chunks pequenos (200-400 caracteres) permitem recuperação precisa de informações específicas mas correm o risco de perder o contexto necessário para interpretar aquela informação. Chunks grandes (1000-2000 caracteres) preservam contexto mas podem incluir ruído e diluir a similaridade semântica.

O overlap entre chunks (ex: 100-200 caracteres) garante que frases importantes cortadas pelo limite apareçam em pelo menos dois chunks consecutivos, reduzindo a perda de informação nas fronteiras. Sem overlap, um conceito que começa no final de um chunk e termina no início do próximo nunca será recuperado por completo.

A estratégia de chunking mais comum é o RecursiveCharacterTextSplitter, que tenta primeiro quebrar por parágrafos ($\n$), depois por linhas ($\n$), depois por frases (.), e por último por caracteres, garantindo que as quebras ocorram nos limites naturais do texto sempre que possível.

3. Diferença entre Fine-tuning e RAG

Fine-tuning e RAG são duas abordagens distintas para adaptar LLMs a conhecimentos específicos.

Fine-tuning: o modelo base é treinado adicionalmente (com pesos atualizados) em um dataset específico do domínio. Isso altera permanentemente os parâmetros do modelo, incorporando o novo conhecimento a seus pesos internos. Vantagens: baixa latência na inferência (uma única chamada ao modelo), não depende de sistema externo de retrieval. Desvantagens: caro computacionalmente, precisa ser refeito quando os dados mudam, pode sofrer de catastrophic forgetting (esquece conhecimento anterior), e requer GPU para treinar.

RAG (Retrieval-Augmented Generation): o modelo base permanece congelado. Um sistema externo de recuperacao (retriever) busca documentos relevantes em uma base de conhecimento, e o LLM gera a resposta condicionada a esses documentos. Vantagens: dados atualizaveis sem re-treinar, mais barato (sem GPU de treino), fontes citaveis e verificaveis, menor risco de alucinacao. Desvantagens: maior latencia (retrieval + geracao), depende da qualidade do retriever, custo de manutencao da base vetorial.

Na pratica, as abordagens sao complementares: RAG e ideal para conhecimento factual atualizavel frequentemente (ex: documentacao de API, noticias), enquanto fine-tuning e melhor para adaptar estilo, tom e comportamento geral do modelo (ex: tornar o modelo especialista em linguagem juridica ou medica).

4. Reducao de Alucinacao em LLMs

Alucinacao em LLMs ocorre quando o modelo gera informacoes plausiveis mas factualmente incorretas ou nao suportadas pelo contexto. As principais estrategias para reduzir alucinacao:

1. RAG com ancoragem no contexto: fornecer documentos relevantes no prompt e instruir o modelo explicitamente a responder APENAS com base no contexto fornecido, com fallback ('Nao encontrado no corpus') quando a resposta nao estiver presente.
2. Prompt engineering: usar temperature baixa (0.0-0.3) para reduzir criatividade excessiva, e incluir instrucoes claras de citacao de fontes.
3. Chunking adequado: chunks muito pequenos fragmentam a evidencia necessaria, fazendo com que o LLM complete informacoes faltantes com 'chutes' plausiveis.
4. Retrieval de qualidade: um retriever que retorna documentos irrelevantes ou incompletos aumenta a probabilidade de alucinacao, mesmo com um prompt bem formulado.
5. Validacao pos-geracao: verificar se as afirmacoes geradas estao de fato presentes nos documentos recuperados (factuality check).
6. Modelos menores e mais antigos (GPT-3.5, Llama-2) tendem a alucinar mais que modelos recentes (GPT-4o, Claude 4, Llama-3.1), que tem melhor capacidade de seguir instrucoes de ancoragem ao contexto.